

Denarii Dolor — Unit Test Plan: Transaction Validation

Product	Denarii Dolor (Android personal budget tracker)
Unit under test	<code>ValidateTransactionUseCase</code> (domain/usecase), supported by <code>Validators</code> and <code>Money.parseToCents</code> (util)
Test class	<code>app/src/test/java/com/denariidolor/ValidateTransactionUseCaseTest.kt</code> (plus <code>ValidatorsTest</code> , <code>MoneyTest</code>)
Test level / type	Unit (JVM, no Android framework), black-box and white-box functional
Author	Joseph Anthony Abbott III
Plan date	2026-09-21

1. Purpose

This plan verifies that Denarii Dolor refuses invalid financial data before anything is written to the database. That covers the course's **validation** requirement: invalid amounts, blank descriptions, invalid dates, and expenses exceeding budget thresholds. It also covers the transfer rules and reference checks that protect account balances.

`ValidateTransactionUseCase` runs before every add and every edit of a transaction. `AddTransactionUseCase` and `UpdateTransactionUseCase` both call it, and nothing is saved unless it returns `Result.success`. If it passes bad data, account balances and reports become wrong. Because every save goes through it, it is the highest-value unit to test in isolation.

2. Scope

In scope

Rule	Where enforced
Amount must be greater than 0 cents	<code>Validators.isValidAmount</code>
Amount text: at most 2 decimals and within range; blanks, letters and exponents rejected	<code>Money.parseToCents</code>
Description must not be blank	<code>Validators.isValidDescription</code>

Rule	Where enforced
Date must be a positive epoch timestamp	<code>Validator.s.isValidDateEpoch</code>
Account and category IDs must be positive and must exist	<code>ValidateTransactionUseCase</code> + repositories
A transfer needs a destination that differs from the source account and exists	<code>ValidateTransactionUseCase</code>
An expense must not push the category's month total over its budget limit (the limit itself is allowed)	<code>ValidateTransactionUseCase</code> + <code>BudgetRepository</code> / <code>TransactionRepository</code>
When editing, the transaction's own previous amount is excluded from the month total	<code>excludeTransactionId</code>
Budgets apply only to expenses; categories without a budget are unlimited	<code>ValidateTransactionUseCase</code>
A repository or database error becomes a <code>Result.failure</code> , not a crash	<code>runSuspendedCatching</code>
Name length and percentage bounds; inverted search ranges	<code>Validator.s</code>

Out of scope for this plan (covered by other suites):

- The Room SQL itself: covered by the instrumented `TransactionDaoTest`.
- UI error messages: covered by the Compose instrumented tests.
- PIN and security logic: covered by `SecurityProfileServiceTest`.

3. Unit under test

Figure 1. The unit under test (source view). The checks run in order and return at the first failure. The cheap field checks come before any repository lookups.

```

32 class ValidateTransactionUseCase @Inject constructor(
33     private val categoryRepository: CategoryRepository,
34     private val accountRepository: AccountRepository,
35     private val budgetRepository: BudgetRepository,
36     private val transactionRepository: TransactionRepository
37 ) {
38     suspend operator fun invoke(transaction: TransactionEntity): Result<Unit> {
39         if (!Validators.isValidAmount(transaction.amountCents)) return failure("Invalid amount")
40         if (!Validators.isValidDescription(transaction.description)) return failure("Description cannot be blank")
41         if (!Validators.isValidDateEpoch(transaction.dateEpochMillis)) return failure("Invalid date")
42         if (transaction.accountId <= 0L) return failure("Invalid account")
43         if (transaction.categoryId <= 0L) return failure("Invalid category")
44         if (transaction.type == TransactionType.TRANSFER) {
45             val transferAccountId = transaction.transferAccountId ?: return failure("Transfer destination is required")
46             if (transferAccountId == transaction.accountId) return failure("Transfer destination must be different")
47         }
48         return runSuspendCatching { checkReferencesAndBudget(transaction) }.getOrElse { Result.failure(it) }
49     }
50
51     private suspend fun checkReferencesAndBudget(transaction: TransactionEntity): Result<Unit> {
52         if (categoryRepository.getById(transaction.categoryId) == null) return failure("Category not found")
53         if (accountRepository.getById(transaction.accountId) == null) return failure("Account not found")
54         val transferAccountId = transaction.transferAccountId
55         if (transaction.type == TransactionType.TRANSFER &&
56             transferAccountId != null &&
57             accountRepository.getById(transferAccountId) == null
58         ) {
59             return failure("Transfer destination account not found")
60         }
61         if (transaction.type == TransactionType.EXPENSE) {
62             val budget = budgetRepository.getByCategoryId(transaction.categoryId)
63             if (budget != null) {
64                 val (start, end) = monthBounds(transaction.dateEpochMillis)
65                 val spent = transactionRepository.getExpenseTotalForCategory(
66                     categoryId = transaction.categoryId,
67                     startInclusive = start,
68                     endInclusive = end,
69                     excludeTransactionId = transaction.id
70                 )
71                 if (spent + transaction.amountCents > budget.monthlyLimitCents) {
72                     return Result.failure(IllegalStateException("Budget threshold violated"))
73                 }
74             }
75         }
76         return Result.success(Unit)

```

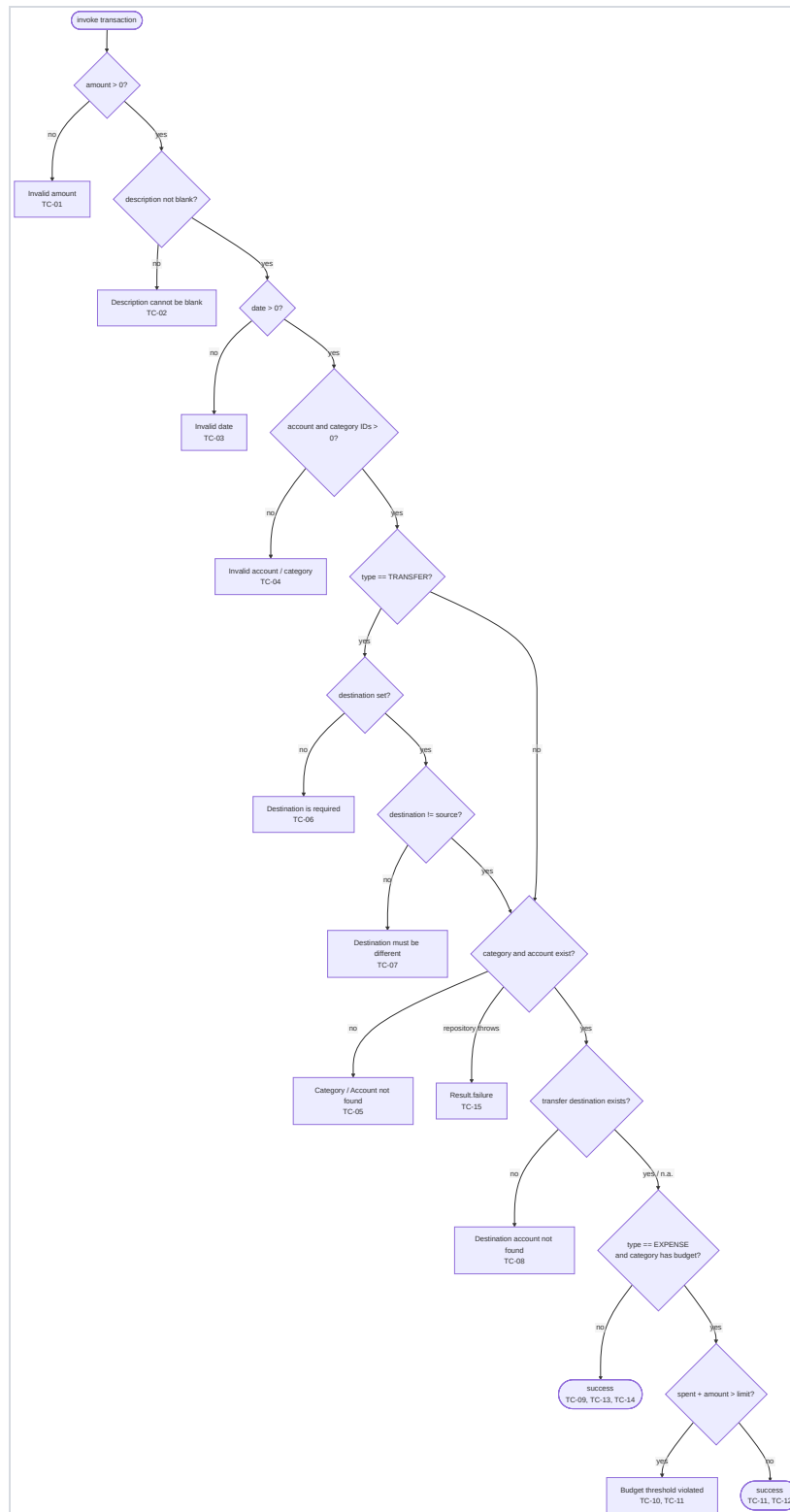
Figure 2. Supporting helpers (source view).

```
Validators.kt / Money.kt — supporting validation helpers

21 object Validators {
22     fun isValidAmount(amountCents: Long): Boolean = amountCents > 0
23
24     fun isValidDescription(description: String): Boolean = description.isNotBlank()
25
26     fun isValidDateEpoch(epochMillis: Long): Boolean = epochMillis > 0
27
28     const val MAX_NAME_LENGTH = 50
29     private const val MAX_PERCENT = 100
30
31     :
32
33 /** Money is stored and computed as whole cents (`Long`); these helpers are the only conversions to and from text. */
34 object Money {
35     private const val SCALE = 2
36     private const val CENTS_PER_UNIT = 100.0
37     private val MAX_INPUT = BigDecimal("9999999999.99")
38     private val DECIMAL = Regex("^-?\\d+(\\.\\d+)?\$")
39
40     /** Parses user input like `12`, `12.5`, `$1,234.56`; returns null for blanks, >2 decimals, or out-of-range values. */
41     fun parseToCents(text: String): Long? {
42         val cleaned = text.trim().replace(",", "").let { if (it.startsWith("-\$")) "-" + it.drop(2) else it.removePrefix("\$") }
43         if (!DECIMAL.matches(cleaned)) return null
44         val value = cleaned.toBigDecimal()
45     }
46 }
```

3.1 Decision flow and test-case coverage

Each branch below is labelled with the test case(s) that exercise it.



4. Test approach

- **Framework:** JUnit 4.13.2 on the JVM. Suspend functions are called with `kotlinx.coroutines.runBlocking<Unit>`, which guarantees each test method returns `void`.

- **Isolation:** the four repository dependencies are replaced by in-memory **test doubles** (FakeCategoryRepository, FakeAccountRepository, FakeBudgetRepository, FakeTransactionRepository), so no database, emulator or Android framework is involved. FakeTransactionRepository.expenseTotalCents lets a test set the month's spending directly. lastExcludedTransactionId records which ID the use case excluded.
- **Techniques:**
 - **Equivalence partitioning:** valid vs. invalid classes for each field.
 - **Boundary value analysis:** amount 0 / -1 / 1; budget exactly at the limit vs. one cent over.
 - **Branch coverage:** every return failure(...) in Figure 1 is reached by at least one test.
 - **Error guessing:** a repository that throws.
- **Structure:** each test follows Arrange-Act-Assert and asserts the **exact** failure message. The UI shows that message to the user, so it is part of the contract.
- **Money:** all amounts are Long cents. For example, 10_000 = \$100.00.

Figure 3. The test fixture: the class under test is wired to fakes, and TestData provides the reference data.

```

36 class ValidateTransactionUseCaseTest {
37     private val transactions = FakeTransactionRepository()
38     private val budgets = FakeBudgetRepository(listOf(BudgetEntity(id = 1, categoryId = 4, monthlyLimitCents = 10_000)))
39     private val useCase = ValidateTransactionUseCase(
40         FakeCategoryRepository(TestData.categories),
41         FakeAccountRepository(TestData.accounts),
42         budgets,
43         transactions
44     )
45
46     :
137 object TestData {
138     val categories = listOf(
139         CategoryEntity(id = 1, name = "General Expense"),
140         CategoryEntity(id = 2, name = "General Income"),
141         CategoryEntity(id = 3, name = "Transfer"),
142         CategoryEntity(id = 4, name = "Groceries")
143     )
144     val accounts = listOf(
145         AccountEntity(id = 1, name = "Cash", balanceCents = 0L),
146         AccountEntity(id = 2, name = "Savings", balanceCents = 0L),
147         AccountEntity(id = 3, name = "Checking", balanceCents = 0L)
148     )
149
150     fun expense(id: Long = 0, amountCents: Long = 2_000, categoryId: Long = 4, accountId: Long = 1) = TransactionEntity(
151         id = id,
152         type = TransactionType.EXPENSE,
153         description = "Coffee",
154         amountCents = amountCents,
155         categoryId = categoryId,
156         accountId = accountId,
157         dateEpochMillis = System.currentTimeMillis()
158     )

```

Fixture data used by every case

Item	Value
Categories	1 General Expense · 2 General Income · 3 Transfer · 4 Groceries

Item	Value
Accounts	1 Cash · 2 Savings · 3 Checking
Budget	Category 4 (Groceries), limit 10 000 ¢ (\$100.00)
Default transaction	<code>TestData.expense()</code> : EXPENSE, "Coffee", 2 000 ¢ (\$20.00), category 4, account 1, date = now

5. Test cases

ID	Test method	Input (changes from the default expense)	Expected result	Technique
TC-01	<code>rejectsZeroOrNegativeAmount</code>	amount = 0; amount = -1	failure "Invalid amount" (both)	Boundary
TC-02	<code>rejectsBlankDescription</code>	description = " "	failure "Description cannot be blank"	Equivalence
TC-03	<code>rejectsNonPositiveDate</code>	date = 0	failure "Invalid date"	Boundary
TC-04	<code>rejectsNonPositiveAccountOrCategoryIds</code>	accountId = 0; categoryId = 0	failures "Invalid account" and "Invalid category"	Boundary
TC-05	<code>failsWhenCategoryOrAccountMissing</code>	categoryId = 99; accountId = 99	failures "Category not found" and "Account not found"	Equivalence
TC-06	<code>transferRequiresDestination</code>	TRANSFER, destination = null	failure "Transfer destination is required"	Equivalence
TC-07	<code>transferFailsWhenDestinationMatchesSource</code>	TRANSFER 1 → 1	failure	Equivalence
TC-08	<code>transferFailsWhenDestinationAccountMissing</code>	TRANSFER 1 → 99	failure "Transfer destination account not found"	Equivalence
TC-09	<code>validTransferPasses</code>	TRANSFER 1 → 2, category 3	success	Positive path
TC-10	<code>expenseFailsWhenBudgetExceeded</code>	spent 9 000 ¢ + 2 000 ¢ > 10 000 ¢	failure "Budget threshold violated"	Equivalence
TC-11	<code>expenseExactlyAtLimitIsAllowed</code>	spent 8 000 ¢ + 2 000 ¢ = limit; + 2 001 ¢	success at the limit; failure one cent over	Boundary
TC-12	<code>editedExpenseExcludesItsOwnPreviousAmountFromBudget</code>	id = 7, spent 8 000 ¢ + 2 000 ¢	success; repository asked to exclude ID 7	White-box
TC-13	<code>budgetOnlyLimitsExpenses</code>	INCOME 5 000 ¢ in budgeted category, spent 9 000 ¢	success	Equivalence
TC-14	<code>expenseWithoutBudgetIsNotLimited</code>	category 1 (no budget), 500 000 ¢, spent 1 000 000 ¢	success	Equivalence
TC-15	<code>repositoryErrorBecomesFailureInsteadOfCrash</code>	category repository throws "database closed"	<code>Result.failure</code> carrying that message; no exception escapes	Error guessing

ID	Test method	Input (changes from the default expense)	Expected result	Technique
TC-16	<code>ValidatorsTest.amountAndDescriptionValidation</code>	1 000 / 0 / -1 cents; "Rent" / " "	true / false / false; true / false	Boundary
TC-17	<code>ValidatorsTest.rangeValidationRejectsInvertedRanges</code>	min > max; start > end; min < max	false; false; true	Equivalence
TC-18	<code>ValidatorsTest.nameAndPercentValidation</code>	names: "Groceries", " ", 51 chars; percents: 80 / 0 / 101	true, false, false; true, false, false	Boundary
TC-19	<code>MoneyTest.parsesCommonInputsToCents</code>	12, 12.5, " 12.05 ", \$1, 234.56, -25.50, 12.000	1200, 1250, 1205, 123456, -2550, 1200	Equivalence
TC-20	<code>MoneyTest.rejectsInvalidInputs</code>	"", abc, 1.005, 1e3, 9999 9999999	null for each	Boundary

TC-01 - TC-04, TC-06, TC-09 and TC-13 - TC-15 were **added while writing this plan**. They close the gaps found when the existing tests were mapped to the decision flow (see the change summary document). The other cases already existed and were kept as they were.

6. Requirements traceability

Course requirement / rule	Test cases
Prevent invalid amounts	TC-01, TC-16, TC-19, TC-20
Prevent blank descriptions	TC-02, TC-16
Prevent invalid dates	TC-03
Prevent expenses exceeding budget thresholds	TC-10, TC-11, TC-12, TC-13, TC-14
Referential integrity (accounts, categories, transfers)	TC-04 - TC-09
Duplicate/invalid names, percentage bounds (reference data)	TC-18 (duplicates: <code>ManageReferenceDataUseCaseTest</code>)
Search input validation	TC-17
Robustness / no crash on data-layer errors	TC-15

7. Environment

Item	Value
OS	Developer workstation (Linux) and GitHub Actions <code>ubuntu-latest</code>
JDK	17 (Temurin)
Build	Gradle 8.11.1, Android Gradle Plugin 8.7.3, Kotlin 2.0.21
Libraries	JUnit 4.13.2, <code>kotlinx-coroutines-test</code>
Coverage	JaCoCo via <code>enableUnitTestsCoverage = true</code>

How to execute

```
./gradlew testDebugUnitTest --tests "com.denariidolor.ValidateTransactionUseCaseTest" \
--tests "com.denariidolor.ValidatorsTest" \
--tests "com.denariidolor.util.MoneyTest"
./gradlew testDebugUnitTest createDebugUnitTestCoverageReport # whole suite + coverage
```

Outputs:

- HTML test report: `app/build/reports/tests/testDebugUnitTest/index.html`
- JUnit XML: `app/build/test-results/testDebugUnitTest/`
- Coverage report: `app/build/reports/coverage/test/debug/index.html`

8. Entry and exit criteria

Entry: the project compiles (`assembleDebug`); ktlint and detekt report 0 findings; the fixture data matches §4.

Exit (pass):

1. All 20 test cases pass, with 0 failures and 0 errors.
2. The full unit suite passes, so there are no regressions.
3. Every `return failure(...)` branch in `ValidateTransactionUseCase` is executed. This is checked in the JaCoCo report for the class, with a target of ≥ 90 % line coverage.
4. CI's *verify* job is green on `main`.

Suspension: a compile failure or failing fixture. Fix it first; test results are not recorded until the suite runs cleanly.

9. Defect handling

A failing case is triaged as either a **product defect** (fix the code, keep the test) or a **test defect** (wrong expectation or setup; fix the test and record why). Every fix is re-run against the full suite and recorded in `history.md` and in the change-summary document.

10. Roles and schedule

Activity	Owner	When
Write the plan and the test scripts	Developer (J. Abbott)	2026-09-21
Execute locally and collect the reports	Developer	2026-09-21
Re-run in CI on push to <code>main</code>	GitHub Actions <code>ci.yml</code>	Every push / PR
Review the results and sign off	Developer / instructor	After a green run